

架构设计文档

对应版本: 平台 v1.2 (2026-09-27 全面复盘后基于源码现状更新)
本文所有模块、路径与数据均以当前代码库为唯一事实来源。

1. 总体架构

平台采用五层分层架构:

L1 前端层	Vue 3.4 + Vite 5 + Three.js 0.166 Element Plus + Pinia + vue-router(Hash) + vue-i18n
L2 接入层	Vite Dev Proxy / Nginx /api/v1 → FastAPI(8000); 公网经 cpolar HTTPS 隧道
L3 后端服务层	FastAPI 0.104 + Pydantic v2 16 个路由模块 / 118 个端点 (含 2 个系统端点); 薄壳编排
L4 算法层	algorithm_model (独立可安装包) NURBS 整车建模 / freeform 自由曲面 / surface_quality 质量评估与优化 / storyboard 分镜
L5 基础设施层	SQLite (开发) + SQLAlchemy 2.0 (11 张 ORM 表) 文件存储; 可选 Redis; Docker Compose 部署

关键设计决策:

- 算法与 Web 解耦:** `algorithm_model/` 是独立 Python 包, 有自己的入口 (`main.py` / `api.py`)、依赖与测试; 后端只做编排。
- 双几何管线并行:**
 - mesh 管线 (trimesh 参数化网格) —— 前端实时预览、GLB/STL 导出;
 - NURBS 管线 (自研 NURBS 核心 + STEP writer) —— STEP 工程输出、G0/G1/G2 A 级曲面分析。
- 能力真实降级:** 可选能力 (PyTorch、Ollama、Redis、CLIP) 缺失时返回 503 或自动降级, 绝不伪装成功。

4. **安全纵深**：JWT 鉴权、Fernet 加密 API Key、生产 SECRET_KEY fail-fast、统一安全响应头、CORS 白名单。

2. 前端结构

源码根： `src/` 。 10 个路由页面（Hash 模式）：

路由	页面	核心职责
<code>/</code>	Dashboard	平台总览与功能入口
<code>/designer</code>	Designer	AI 参数化设计器：参数面板 + 3D 实时渲染
<code>/projects</code>	Projects	项目列表
<code>/projects/:id</code>	ProjectDetail	项目详情与模型管理
<code>/deep-learning</code>	DeepLearning	深度学习设计器：训练批次、风格/生成式设计
<code>/quality</code>	Quality	质量检查与报告列表
<code>/deliver</code>	Deliver	工程交付与数据交接
<code>/demo</code>	Demo	功能演示
<code>/login</code>	Login	登录（密码 + 微信扫码/公众号授权），公开页
<code>/account</code>	Account	账户与 API Key 管理

核心目录：

- `components/`：Car2D、Car3D、MarkdownRenderer、TechSelectionMatrix
- `stores/`：Pinia —— auth / designer / project / ui
- `api.js`：统一 axios 实例（baseURL `/api/v1`），按域导出 `projectAPI` ... `bayesAPI`
- `config/carPresets.js`：品牌车型预设（5 品牌 19 车型）
- `utils/`：carImageManager、imageGenerator、llm、useCarSessionImage
- `i18n.js`：中英双语

路由全局守卫：无 token 访问受保护页面 → 跳 `/login` 并携带回跳地址。

3. 后端结构

后端根： `backend/` ，应用入口 `app/main.py`（`create_app()` 装配），启动脚本 `start.py`。

3.1 路由模块 (16 个模块共 116 个端点: 113 个 @router + keys_router 3 个; main.py 另有 2 个系统端点, 合计 118 个 HTTP 端点)

模块文件	最终路径前缀	端点 数	职责
project.py	/api/v1/projects	5	项目 CRUD
model.py	/api/v1/models	4	模型文件上传/查询/删除
build.py	/api/v1/build	6	模型构建/重建/批量/缓存
car.py	/api/v1/car	6	车身与部件生成、参数、导出
modify.py	/api/v1/modify	15	NURBS 曲面、参数、测量、历史
export.py	/api/v1/export	4	模型导出与下载
variant.py	/api/v1/variants	7	变体/版本/对比/回滚
workflow.py	/api/v1/workflows	7	工作流与步骤、执行
quality.py	/api/v1 (quality/topology/data 等)	8	质量检查、拓扑优化、数据交接、文件下载
training.py	/api/v1/ai	14	PyTorch 训练任务、样本批次、分析端点
ai.py	/api/v1/ai	3	NURBS 专家对话、Ollama 模型/健康
bayes.py	/api/v1/bayes	7	贝叶斯优化会话全生命周期
texture.py	/api/v1/texture	3	参数化纹理分析与应用
import_export.py	/api/v1/import-export	11	导入→改参→导出全链路会话
llm_proxy.py	/api/v1/llm	4	多 LLM 提供商统一代理
auth.py	/api/v1/auth 、 /api/v1/api-keys	12	注册/登录/微信授权 (9) 、API Key 管理 (3)

逐端点明细见 [api_reference.md](#)。

3.2 领域模块 (backend/app/)

模块	职责
bayes_optimizer.py	高斯过程 (RBF) + EI/UCB 贝叶斯引擎 (纯 numpy/scipy)
brand_knowledge.py	品牌/车型知识库查询、语义匹配、相似检索
cad_importer.py	CAD 文件 (STL/STEP 等) 导入与点云分析、降级兜底
car_generator.py	基于参数配置的 NURBS 车身生成
nurbs.py	NURBS 数据结构与运算
texture_analyzer.py	纹理几何/语义特征提取
clip_semantics.py	CLIP 语义特征 (可选, 缺失自动降级)
session_store.py	通用会话存储
security.py	JWT 签发/校验、依赖注入 (get_current_user / get_optional_user)
config.py	pydantic-settings 配置; 生产 SECRET_KEY fail-fast
database.py	引擎、SessionLocal、11 张 ORM 表
schemas.py	Pydantic 请求/响应模型

3.3 配置与插件

- backend/config/automotive_parameters.json : 造型规范参数 (整车尺寸 / 车身部件 / 造型角度 / A 级曲面参数 / 比例参数)
 - backend/config/brand_design_knowledge.json : 5 品牌 19 车型知识库
 - backend/rhino_plugin/ : Rhino 插件 (命令 + 语义 API) ; 项目根 rhino/ 为品牌 DNA 面板
 - scripts/llm_server.py + Modelfile : 本地 LLM 专家服务 (Docker / 本地双模)
-

4. 算法层结构 (algorithm_model/)

子包	内容
car_modeling/	参数化整车：car_params (22 维参数)、body (截面环网格)、body_nurbs、body_ends* (端体 G1/附着式)、glass/wheels/lights/grille/mirrors/seams/trim、assembler (装配与统计)、blending (三区段 + tumblehome)、continuity_checker、sop_checklist
freeform/	NURBS 核心 (nurbs_core)、自由曲面、扫掠、圆角、STEP writer; Cython 加速 (_nurbs_cy.pyx)
surface_quality/	曲率、G0/G1/G2 连续性、反射线、质量分级、模拟退火光顺优化; Cython (_quality_cy.pyx)
storyboard/、 storyboard_viewer/	分镜生成 (模板) 与 Markdown/HTML 渲染
examples/	17 个示例 (整车、NURBS STEP、G1 连续性、SOP 报告等)

对外统一入口：api.py (build_car / get_car_stats / evaluate_surface / optimize_surface / make_storyboard / render_storyboard / run_full_pipeline) 。

5. 数据模型（11 张 ORM 表）

表	模型类	关键字段
projects	Project	name, description, status
model_files	ModelFile	project_id, filename/filepath, file_type, status, params_json, car_data_json
workflows	Workflow	project_id, name, type, status
workflow_steps	WorkflowStep	workflow_id, model_id, step_name/type, status, progress, input/output, error
quality_reports	QualityReport	project_id, model_id, overall_score, passed, report_data, report_path
parameter_sets	ParameterSet	project_id, name, params
model_variants	ModelVariant	model_id, name, parent_variant_id, params_json, car_data_json
users	User	email(唯一), username, password_hash, wechat_unionid/openid, is_active/is_admin
api_keys	ApiKey	user_id, provider, key_encrypted; (user_id, provider) 唯一
parameter_records	ParameterRecord	name(唯一), value
training_tasks	TrainingTask	user_id, name, dataset, config_json, status, progress, metrics_json, logs

关系链：Project 1—N ModelFile / Workflow / QualityReport；Workflow 1—N WorkflowStep；ModelFile 1—N ModelVariant（含自引用 parent）；User 1—N ApiKey。

6. 关键数据流

6.1 AI 参数化设计闭环

Designer 页面 → carAPI.generate(params)
 → car_generator 参数化生成（mesh 预览 + 数据）
 → ModelFile 持久化（params_json / car_data_json）
前端 Car3D 实时渲染；变体经 variantAPI 保存/对比/回滚

6.2 导入→改参→导出全链路

文件上传 → `cad_importer` 解析（失败降级点云分析）→ `import_export` 会话
→ GET `params`（参数树/分组）→ PUT `params`（覆盖）
→ GET `preview`（3D 预览）→ POST `export`（多格式）→ `download`

6.3 贝叶斯优化→训练联动

POST `/bayes/sessions` → `suggest`（GP+EI/UCB）
→ `/ai/evaluate-quality`（或人工）评分 → `observe` 回填，迭代
→ `/samples` 导出（`feature_order` + `features/score`）
→ `/ai/train` 后台 PyTorch 训练，`/ai/tasks` 轮询状态

6.4 微信登录

Login → `/auth/methods` 探测能力
→ 开放平台扫码（`/auth/wechat/qr` → `callback`）
或公众号网页授权（`/auth/mp/authorize` → 前端轮询 `/auth/mp/poll` → `callback`）
→ JWT（`evoai_token`）

7. 横切关注点

- **鉴权**：JWT（PyJWT），密码哈希存储；敏感端点 `Depends(get_current_user)`，半开放端点 `get_optional_user`。
- **密钥管理**：用户 LLM API Key 以 Fernet 对称加密入库；cpolar authtoken / appsecret 只进 `.env`，禁止进入命令载荷。
- **安全响应头**：`main.py` 中间件统一注入 X-Content-Type-Options / X-Frame-Options / CSP / Referrer-Policy / Permissions-Policy / Cache-Control；去除 `server: uvicorn`。
- **CORS**：DEBUG 放行，生产显式白名单；禁止 ACAO * + credentials 组合（含 ASGI 校正中间件）。
- **异常处理**：StarletteHTTPException / RequestValidationError / Exception / 405 统一处理，错误响应同样补齐 CORS。

8. 部署架构

- **本地开发**：Vite 5173 + FastAPI 8000 + cpolar 4040（HTTPS 隧道，公网域名）。
长期服务以独立顶层进程启动（`.trae/skills/launch-services-detached`）。

- **容器化**: `deploy/docker-compose.yml` (backend / frontend / nginx; 必填环境变量校验) , 镜像由 `deploy/*.Dockerfile` 构建, `deploy/nginx.conf` 与 `default.conf.template` 反代。
- **LLM 服务**: `docker-compose.llm.yml` + `Dockerfile.llm` (Ollama/专家模型) , 支持 Docker / 本地双模自动切换。

9. 质量基线

测试套件	用例数	结果
algorithm_model pytest (tests/)	200	全过
algorithm_model 一站式自检 (test_all.py)	5 大模块	全过
backend pytest (tests/)	178	全过
frontend vitest	111	全过
合计	489+	全绿

运行方式见根 README「测试」一节。